Job Posting Acceptance Tests

Purpose of AI Use

Review existing job-posting tests and implement additional tests for the creation, editing, and deletion flows.

AI-Suggested Content

The AI identified that the existing job-posting test file covered only parsing and validation, not the server-side create, edit, or delete operations. It added testable server-side helpers, wired the relevant SvelteKit actions through those helpers, and added tests for:

- Creation: recruiter-only access, invalid form handling, and successful insertion with the recruiter assigned as creator.
- Editing: recruiter authorization, invalid IDs and form values, successful owner updates, and preventing other recruiters from editing a posting.
- Deletion: recruiter authorization, invalid IDs, successful owner deletion, preventing other recruiters from deleting a posting, and missing postings.

Validation

After the creation tests were added, `npm test` passed all 47 tests and `npm run check` reported no errors or warnings.

After the editing tests were added, `npm test` passed all 52 tests and `npm run check` reported no errors or warnings.

After the deletion tests were added, `npm test` passed all 57 tests and `npm run check` reported no errors or warnings.

Decision

Accepted / Modified. The implementation reuses the existing Node test runner and assertion style. The added tests exercise extracted server-side operation logic with stubbed persistence, not full HTTP requests through SvelteKit or a live database.

Reflection

The AI helped identify missing coverage across the job-posting lifecycle and added focused tests while preserving route behavior. Extracting the business operations made authorization, validation, ownership, and persistence outcomes independently testable. These are unit-level acceptance-style tests rather than end-to-end acceptance tests, so they do not verify browser behavior, actual routing/redirect responses, or a real database integration.

Responsible Person

Michael Derocher

Prompt 1

Asked whether the repository already had tests for recruiter-profile job-posting creation.

Response 1

Inspected the test files and found `src/lib/server/job-posting.test.ts`. Its tests cover `parseJobPosting`: trimming input, required fields and valid employment/work-mode choices, rejecting inherited object-property names as choices, field length, valid/malformed company domains, requirements and deadline, salary parsing/ranges, and posting status. They did not cover create-action authorization, persistence, or redirect behavior.

Prompt 2

Asked whether job-posting editing and deletion were covered by tests.

Response 2

Found no edit/delete tests. Read `src/routes/(app)/jobs/[id]/edit/+page.server.ts` and confirmed that it already implemented save and delete actions with recruiter-role checks and database conditions scoped to both the posting ID and `locals.user.id`. Existing tests covered parsing only.

Prompt 3

Requested missing creation-flow tests only, deferring edit and delete coverage.

Response 3

Added `src/lib/server/job-posting-create.ts` with `createJobPosting`, which checks for recruiter role, validates the submitted `FormData` through the existing `parseJobPosting` function, and calls an injected insert function with the parsed fields plus `createdBy: user.id`. Updated `src/routes/(app)/jobs/new/+page.server.ts` to use this helper and map its outcomes to the existing SvelteKit responses: 403 for non-recruiters, 400 with validation errors/values for invalid submissions, and a 303 redirect to `/jobs/{id}` after insertion.

Added `src/lib/server/job-posting-create.test.ts` with tests that verify non-recruiters and invalid forms do not invoke insertion, and that a valid recruiter submission inserts parsed/default values with the authenticated recruiter's ID and returns the created posting ID. `npm test` passed all 47 tests; `npm run check` reported no errors or warnings.

The new creation helper returns explicit outcome objects and leaves database insertion to the callback supplied by the route:

```
if (user.role !== 'recruiter') return { status: 'forbidden' };

const parsed = parseJobPosting(formData);
if (!parsed.ok) return { status: 'invalid', errors: parsed.errors, values: parsed.values };

const id = await insert({ ...parsed.values, createdBy: user.id });
return { status: 'created', id };
```

The route's injected callback performs the Drizzle insert and returns the generated ID. Its tests assert that the callback is not called for forbidden or invalid submissions and that a valid submission includes `createdBy: 'recruiter-1'`, parser defaults (`status: 'open'`, `published: false`), and the other parsed values.

Prompt 4

Asked whether the new tests followed the existing tests' style.

Response 4

Confirmed that the new tests use the repository's `node:test` and `node:assert/strict` conventions and construct test inputs directly with `FormData`. Clarified that the persistence callback is stubbed: these are unit tests of the shared server-side operation, not end-to-end HTTP tests through SvelteKit or tests against a real database.

Prompt 5

Requested comparable acceptance-style coverage for saving edits to job postings.

Response 5

Added `src/lib/server/job-posting-edit.ts` with `editJobPosting`. It rejects non-recruiters, rejects IDs that do not match the route's UUID format, parses and validates form data, and calls an injected update function with the posting ID, the authenticated owner ID, parsed values, and a new `updatedAt` timestamp. A missing update result becomes `not-found`.

Updated the `save` action in `src/routes/(app)/jobs/[id]/edit/+page.server.ts` to use the helper. Its database update remains owner-scoped by both posting ID and creator ID; action outcomes map to the existing 403/404/400 failures, or a 303 redirect to `/jobs/{id}?saved=1`.

Added `src/lib/server/job-posting-edit.test.ts` to test non-recruiter rejection, invalid UUID rejection, invalid form rejection without update, successful owner editing (including the new timestamp), and denial of another recruiter's update without changing the in-memory posting. `npm test` passed all 52 tests; `npm run check` reported no errors or warnings.

The edit helper validates role and UUID before parsing or invoking persistence, then supplies the update callback with the authenticated owner ID and timestamp:

```
if (user.role !== 'recruiter') return { status: 'forbidden' };
if (!UUID.test(postingId)) return { status: 'not-found' };

const parsed = parseJobPosting(formData);
if (!parsed.ok) return { status: 'invalid', errors: parsed.errors, values: parsed.values };

const id = await update(postingId, user.id, { ...parsed.values, updatedAt: new Date() });
return id ? { status: 'updated', id } : { status: 'not-found' };
```

The route callback applies those values only to a row matching both the requested posting and the logged-in recruiter:

```
.set(values)
.where(and(eq(jobPosting.id, id), eq(jobPosting.createdBy, ownerId)))
```

```
.returning({ id: jobPosting.id });
```

The owner test uses an in-memory posting and verifies its title changes to `Updated Software Engineer` and that `updatedAt` is a `Date`. The non-owner test verifies a `not-found` outcome and that the original title stays unchanged.

Prompt 6

Requested acceptance-style tests for deleting job postings.

Response 6

Added `src/lib/server/job-posting-delete.ts` with `deleteJobPosting`. It rejects non-recruiters and malformed UUIDs before persistence, invokes an injected removal callback with the posting ID and authenticated owner ID, and returns `not-found` when no owned posting is removed.

Updated the `delete` action in `src/routes/(app)/jobs/[id]/edit/+page.server.ts` to use the helper. The database delete remains scoped to both posting ID and creator ID; action outcomes retain the existing 403/404 failures and redirect successful deletions to `/jobs?mine`.

Added `src/lib/server/job-posting-delete.test.ts` with tests for non-recruiter access, invalid IDs, deleting an owned posting, refusing to delete another recruiter's posting without changing it, and a missing posting. The tests use an in-memory persistence callback. `npm test` passed all 57 tests; `npm run check` reported no errors or warnings.

The delete helper's role/ID checks and owner-scoped callback are:

```
if (user.role !== 'recruiter') return { status: 'forbidden' };
if (!UUID.test(postingId)) return { status: 'not-found' };

const id = await remove(postingId, user.id);
return id ? { status: 'deleted', id } : { status: 'not-found' };
```

The route callback retains the database ownership predicate and returns `null` when no matching row was deleted:

```
.where(and(eq(jobPosting.id, id), eq(jobPosting.createdBy, ownerId)))
.returning({ id: jobPosting.id });
return posting?.id ?? null;
```

The success test removes the matching owner/posting pair from an in-memory array and asserts the array is empty. The non-owner test asserts `not-found` and verifies that the other

recruiter's posting remains in the array. Additional tests verify that forbidden users and malformed UUIDs never reach the callback, and that a missing row maps to `not-found`.